

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «КиСБ»
Тема: «Псевдослучайные последовательности. Генерация и проверка
простых чисел»

Выполнил:
Студент 4-го курса
Группы АС-66
Савинец М. Д.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: исследовать квадратичный конгруэнтный генератор, оценить свойства последовательностей и реализовать генерацию простых чисел с полиномиальным тестом AKS.

Ход работы

Вариант 11

Часть 1 Генератор псевдослучайных чисел

Постановка и реализация

Цель: исследовать псевдослучайные последовательности и методы проверки простоты, объединив оба этапа в одном консольном приложении на Python. Вся программа находится в lab1.py; запуск без аргументов выполняет демонстрацию и эксперименты.

Вариант 11: квадратичный конгруэнтный генератор с $N=10$, случаи $c=0$ и $c \neq 0$; полиномиальный тест простоты AKS. Все алгоритмы реализованы на Python с использованием стандартной библиотеки. Приложение не вызывает программу на C++, SymPy или готовую функцию проверки простоты.

Формула: $x_{i+1} = (d \cdot x_i^2 + a \cdot x_i + c) \bmod 10$. Параметры a, c, x_0 принадлежат $0 \dots 9$, $d \in 1 \dots 9$. Первым выдаётся x_1 . Последовательность формируется потоково, без сохранения всех чисел в памяти.

```
def next(self) -> int:
    p, x = self.params, self.state
    self.state = (p.d * x * x + p.a * x + p.c) % 10
    return self.state
```

Перебор всех $9 \times 10 \times 10 \times 10 = 9000$ конфигураций подтвердил максимумы:

Случай	Параметры (d, a, c, x_0)	Цикл состояний	Период
$c=0$	(5,2,0,1)	$1 \rightarrow 7 \rightarrow 9 \rightarrow 3 \rightarrow 1$	4
$c \neq 0$	(5,6,1,0)	$0 \rightarrow 1 \rightarrow \dots \rightarrow 9 \rightarrow 0$	10

В обоих случаях предпериод равен нулю. Для $c=0$ состояние $x_0=0$ является неподвижным. Тесты сравнивают поиск цикла с независимым алгоритмом Флойда.

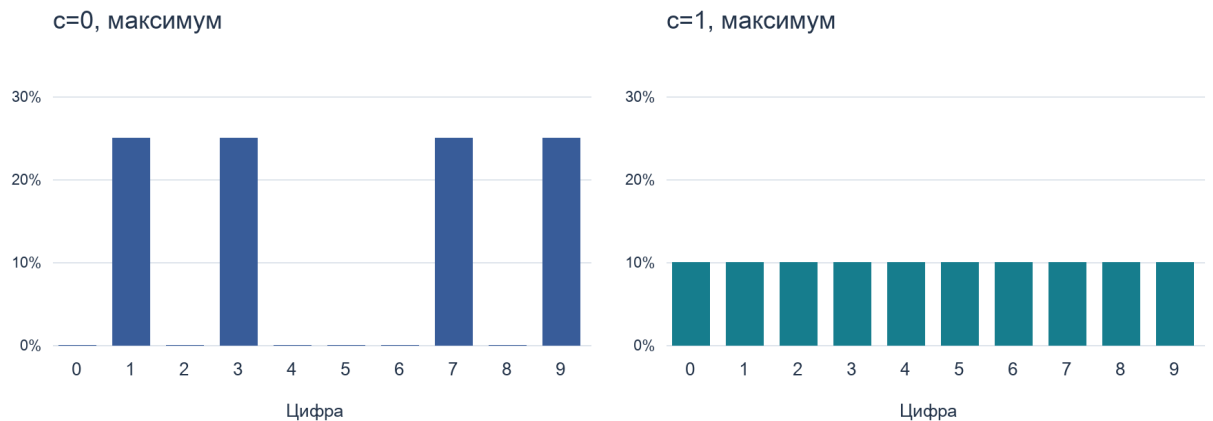
Для любого целого x разность $x^2 - x$ чётна, поэтому $5x^2 \equiv 5x \pmod{10}$. Выбранные функции эквивалентны $7x \bmod 10$ и $x+1 \bmod 10$. Полный период достигается, но преобразование вырождается в линейное на данном маленьком алфавите.

Статистика и влияние параметров

Исследованы шесть конфигураций на длинах 100, 1000 и 10000. Рассчитаны частоты, χ^2 , эмпирическая энтропия, частоты соседних пар и корреляция. В таблице $L=10000$.

Конфигурация	(d, a, c, x_0)	Период	χ^2	Н, бит	ρ_1
$c=0$, максимум	(5,2,0,1)	4	15000	2.0000	0.0001

Конфигурация	(d,a,c,x ₀)	Период	χ^2	H, бит	ρ_1
c=0, x ₀ =0	(5,2,0,0)	1	90000	-0.0000	не определена
c=1, максимум	(5,6,1,0)	10	0	3.3219	0.4544
c=1, x ₀ =7	(5,6,1,7)	10	0	3.3219	0.4545
c=2	(5,6,2,0)	5	10000	2.3219	-0.0001
d=a=c=1	(1,1,1,0)	1	89980	0.0015	не определена



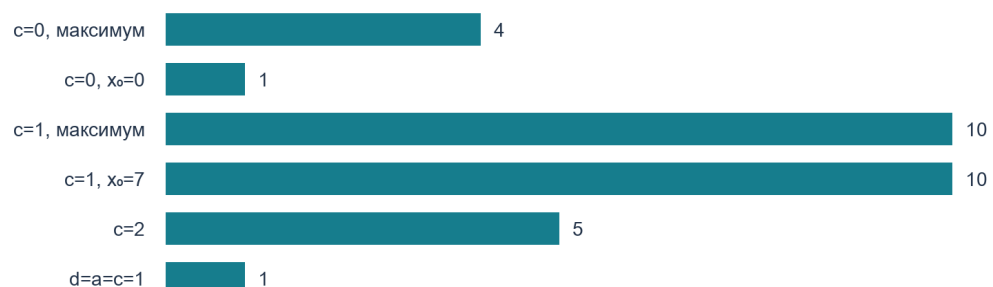
Частоты двух максимальных циклов

Ожидаемая частота цифры $E=L/10$. Использована статистика $\chi^2=\Sigma(O_i-E)^2/E$. Эмпирическая энтропия $H=-\Sigma p_i \cdot \log_2 p_i$; максимум для десяти цифр равен $\log_2 10 \approx 3,32193$ бита.

У выбранного c=0 встречаются четыре цифры и $\chi^2=1,5L$: 150, 1500 и 15000 на трёх длинах. Для полного периода на длинах, кратных десяти, $\chi^2=0$. Это описательные показатели детерминированных последовательностей: p-value и доказательство независимой случайности не заявляются.

Период и зависимость соседних чисел

Период при разных параметрах

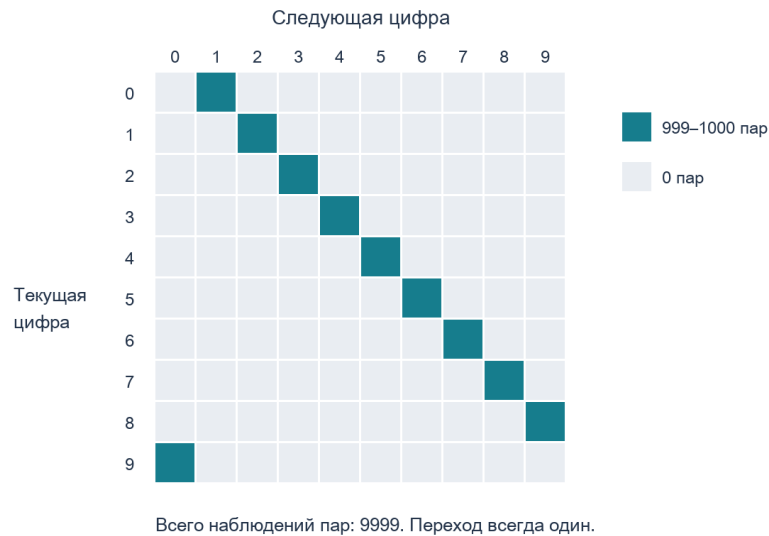


Периоды конфигураций

Изменение начального состояния полного цикла сохраняет период и частоты, но сдвигает фазу. Изменение c=1 на c=2 сокращает период до 5. При c=0 и x₀=0

получается постоянная последовательность; корреляция не определена из-за нулевой дисперсии.

Равные частоты цифр скрывают зависимость



Матрица соседних пар

У полного цикла наблюдаются только 10 из 100 возможных соседних пар. При $L=10000$ всего пар 9999, χ^2 для них равно 89991.01. Следовательно, равные частоты цифр сочетаются с полной предсказуемостью следующей цифры. Один частотный тест или корреляция не заменяют проверку независимости.

Часть 2 Генерация и проверка простых чисел

Полиномиальный тест AKS на Python

Применена уточнённая опубликованная схема AKS [3]. Исключаются точные степени, выбирается r с большим мультипликативным порядком n по модулю r , проверяются НОД и тождества $(X+a)^n \equiv X^n + a \pmod{X^r - 1, n}$. Вердикт детерминированный; вероятностной подмены нет.

Целые границы $B=(n-1).bit_length()=\lceil \log_2 n \rceil$, порядок $>B^2$ и число тождеств $\lceil \sqrt{\phi(r)} \rceil \cdot B$ не меньше необходимых теоретических границ. Целые корни вычисляются через `isqrt`; точные степени проверяются двоичным поиском, без вещественного округления.

Полином представлен списком вычетов. Для ускорения свёртки используется подстановка Кронекера: коэффициенты становятся цифрами большого целого с основанием $Q=256^w$. Выбирается w так, что $Q>r(n-1)^2$, поэтому коэффициенты обычного произведения не создают переносов между разрядами.

```
width = ((r * (n - 1) ** 2).bit_length() + 7) // 8
packed_left = int.from_bytes(
    b''.join(c.to_bytes(width, 'little') for c in left), 'little')
packed_right = int.from_bytes(
    b''.join(c.to_bytes(width, 'little') for c in right), 'little')
raw = (packed_left * packed_right).to_bytes(2 * r * width, 'little')
ordinary = [int.from_bytes(raw[i:i + width], 'little')
    for i in range(0, len(raw), width)]
return [(ordinary[k] + ordinary[k + r]) % n for k in range(r)]
```

После распаковки коэффициенты степеней $k+r$ добавляются к степени k , поскольку $X^r=1$. Эта арифметика отдельно сопоставлена с наивной свёрткой. Возведение в степень выполняется двоичным методом; умножение на $X+a$ использует сдвиг и масштабирование за $O(r)$.

В скане [2] приведена ранняя схема и ошибочный ответ «составное» при $r=p$. Здесь используется исправленная опубликованная форма AKS, корректная в том числе для малых простых. Разрядность приложения ограничена 32 битами, $r \leq 4096$; при нехватке ёмкости возвращается отдельная ошибка.

Генерация простых чисел и новые измерения

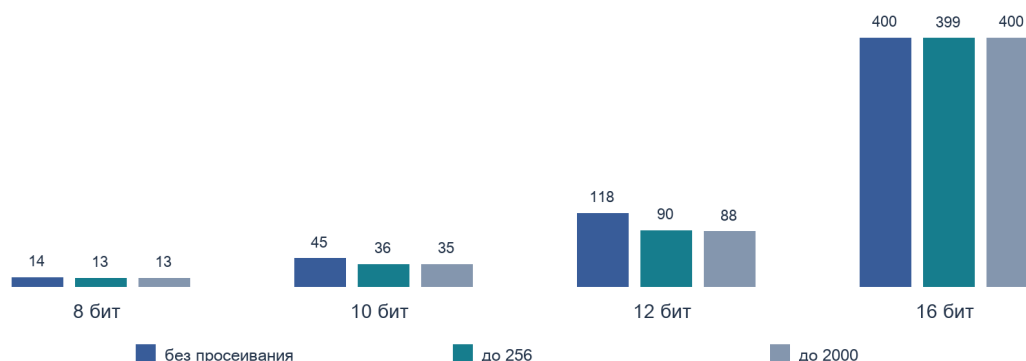
Цифры генератора 0–4 преобразуются в 0, 5–9 — в 1. Из битов собирается начальный кандидат; установка старшего и младшего битов гарантирует нужную разрядность и нечётность, но не создаёт независимой случайности.

Далее проверяются следующие нечётные числа, как разрешено методичкой. По достижении верхней границы поиск возвращается к наименьшему нечётному данной длины. Число попыток ограничено числом различных кандидатов и пользовательским лимитом.

Просеивание выполняется малыми простыми до 256 или 2000; 0 отключает просев. Само малое простое не отбрасывается. Для каждого оставшегося кандидата запускается AKS. Все приведённые времена заново измерены Python-реализацией.

Биты	Просев	Простое	Кандидаты	Отсев	AKS вызовов	Время, мс
8	0	149	4	0	4	14.03
8	256	149	4	3	1	12.84
8	2000	149	4	3	1	12.84
10	0	577	2	0	2	44.60
10	256	577	2	1	1	35.55
10	2000	577	2	1	1	35.10
12	0	2297	1	0	1	117.80
12	256	2297	1	0	1	89.75
12	2000	2297	1	0	1	88.12
16	0	36739	1	0	1	399.68
16	256	36739	1	0	1	399.29
16	2000	36739	1	0	1	400.21

Время поиска, мс · один запуск на конфигурацию



Измерения Python

Дополнительный 32-битный запуск: начальное число 2407784697; найдено простое **2407784719**. Проверено 12 кандидатов, просеиванием отвергнуто 10, AKS отверг 1, вызовов AKS — 2. Время — 35.532 с. Результат независимо подтверждён полным перебором делителей до целого квадратного корня. Этот запуск не входит в график малых размеров.

В таблице по одному запуску на конфигурацию. Время включает проверки и запись журнала, но не построение малой таблицы простых. Оно зависит от нагрузки и не является устойчивой оценкой ускорения. На 12 и 16 битах начальный кандидат уже прост, поэтому просев не отбрасывает ни одного числа.

Проверки, память и ограничения Python

Успешно выполнено **21 автоматический тест unittest**, каждый из которых может проверять много входов.

Все 9000 конфигураций генератора сопоставлены с независимым алгоритмом Флойда.

Упакованное произведение проверено против наивной циклической свёртки на 100 воспроизводимых случайных случаях, включая модуль $2^{32}-1$.

Проверен предельный размер кольца 4096 на максимальных коэффициентах, чтобы выявить переносы при упаковке.

Полиномиальные тождества сопоставлены с независимым раскрытием бинома Ньютона: $n=2\dots24$, $r\in(2, 3, 7)$, $a\in(1, 2, 5)$.

AKS проверен для каждого $n=0\dots300$ по независимому решету Эратосфена; отдельно проверены простые 997 и 1009.

Отвергнуты 341, числа Кармайкла 561, 1105, 1729, 2465, 2821, 6601 и составное число $2^{32}-1$.

Число $1022117=1009\cdot1013$ отвергается именно полиномиальным этапом; недостаточная ёмкость кольца даёт отдельный результат.

Проверены кандидаты для всех длин $2\dots32$ и десяти начальных состояний, просев, журнал CSV, переход через верхнюю границу, лимиты и ошибочные аргументы команд.

Внутри Python используются целые произвольной точности, списки и байтовые строки. Весь объём последовательности не хранится; память полиномов растёт с g и размером коэффициентов.

Python использует динамическую память и не обеспечивает constant-time. Соответствующие ограничения общего README курса не соблюдаются в этой реализации на языке Python. Не заявляются отсутствие выделений в куче, постоянное время операций или аудит побочных каналов.

Границы учебного интерфейса: $0 \dots 2^{32}-1$ для проверки, $2 \dots 32$ бита для генерации, $r \leq 4096$. Криптографические размеры 1024/2048 бит не поддерживаются. Генератор с десятью состояниями непригоден для создания секретных ключей.

Вывод

Обе части варианта 11 реализованы на Python в одном приложении. Максимальные периоды 4 и 10 подтверждены полным перебором. Исследования показали зависимость результатов от параметров и различие между равномерными частотами и независимостью последовательности.

Генератор включён в построение начального кандидата; поиск с просеиванием и AKS формирует подробный журнал. Полиномиальная арифметика и проверка простоты сопоставлены с независимыми реализациями. Время работы измерено заново; результаты старой реализации на другом языке не использованы как измерения Python.

1. Материалы KiSB-2026-autumn, lab1, версия 912d743b68eebfa9ad6b232e55cf6d5944798b6b. Первая часть: постановка, печатная стр. 41; вариант 11, стр. 42–43. Репозиторий (<https://github.com/bstu-iit-information-security/KiSB-2026-autumn/tree/main/lab1>).

2. «Лабораторная работа №1 (ч.2).pdf»: полиномиальный тест, стр. 50–51; генерация, стр. 53–54; варианты, стр. 55. Методичка (<https://github.com/bstu-iit-information-security/KiSB-2026-autumn/blob/main/lab1/Лабораторная%20работа%20№1%20%28ч.2%29.pdf>).

3. M. Agrawal, N. Kayal, N. Saxena. PRIMES is in P. Annals of Mathematics, 160 (2004), 781–793; опубликованная авторами версия, раздел 4. Алгоритм (https://www.cse.iitk.ac.in/users/manindra/algebra/primality_v6.pdf).

Работа подготовлена локально. Для сдачи по правилам курса нужен личный форк, отдельная ветка и Pull Request «[Лаб 1] Савинец Марк — Вариант 11».